



Introduction of NOSQL

The outline

1 What is SQL?

2 Examples of RDBMS

3 What is NOSQL?

4 Comparison between
SQL and NOSQL

5 When to use SQL and
NOSQL?

6 Types of NOSQL

What is SQL?

What is SQL?

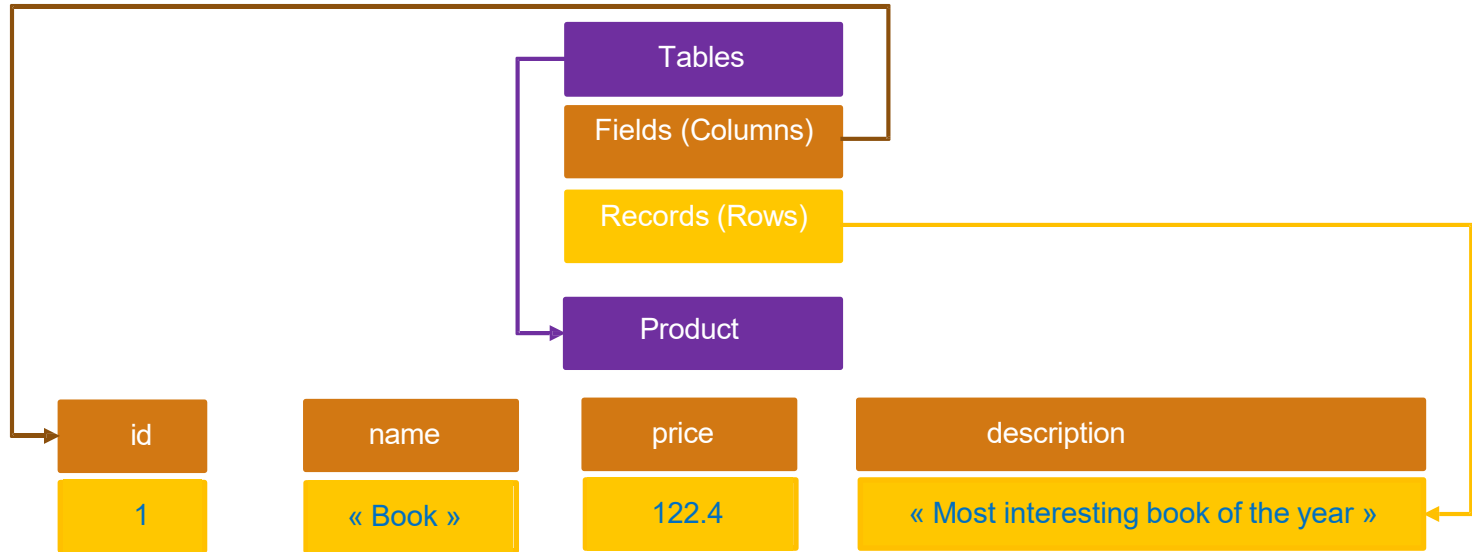
- It stands for **S**tructured **Q**uery **L**anguage. It allows writing database queries to retrieve, insert, update and delete data(**CRUD** operations). Also, the creation of tables, and join queries. For instance,

SELECT id, name, price **from** PRODUCT

- These queries are built using specific key words (ex: select, join), syntax and format using parameters (id, price ..)

The database structure: Relational

- The relational databases support SQL. They are made of set of tables.
- each table contains a set of fields, and many records.
- Using sql, the data must follow a strict schema (set of fields)
- Each new record must have the set of fields defined at the creation of the table. It cannot have more fields!



The database structure: Relational

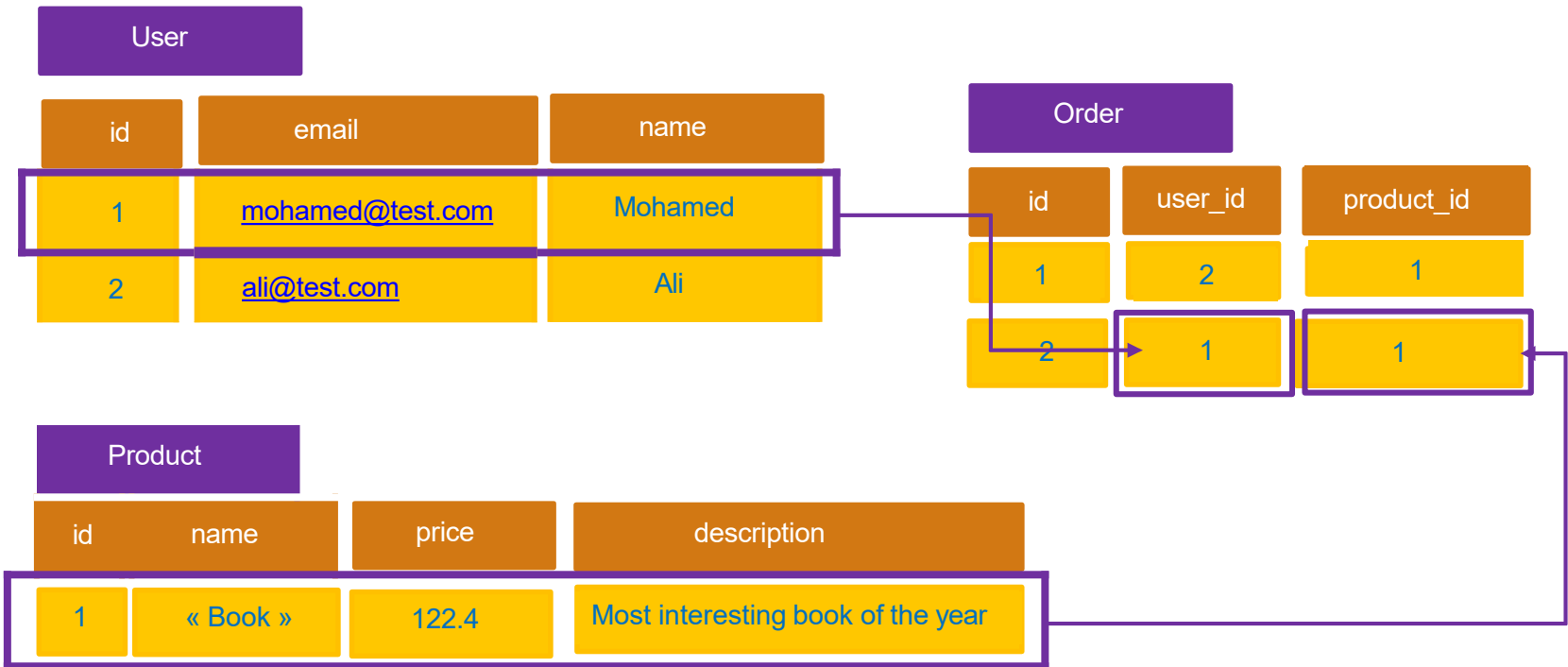
- All records must be normalised to adhere to the same schema (STRICT)

Product			
id	name	price	description
1	« Book »	122.4	« Most interesting book of the year »
2	« Chair »	90.5	« a comfortable chair »
3	...	...	...

ALL records must follow the same schema

The database structure: relations

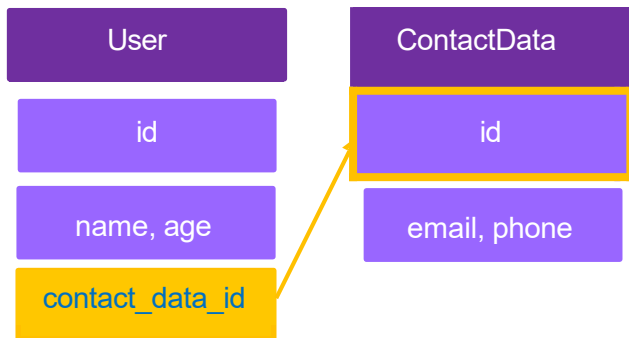
- The relational database has multiple tables that are **RELATED**



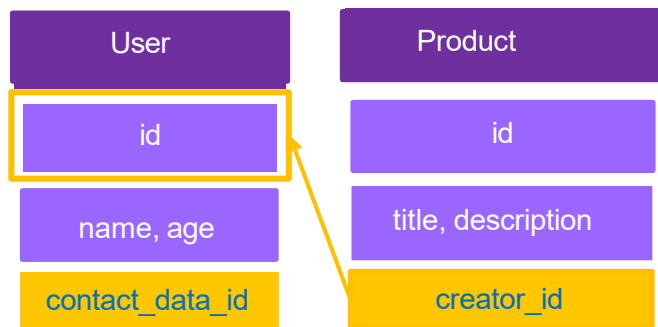
The database structure: types of relations

- The relational database has multiple tables that are **RELATED**

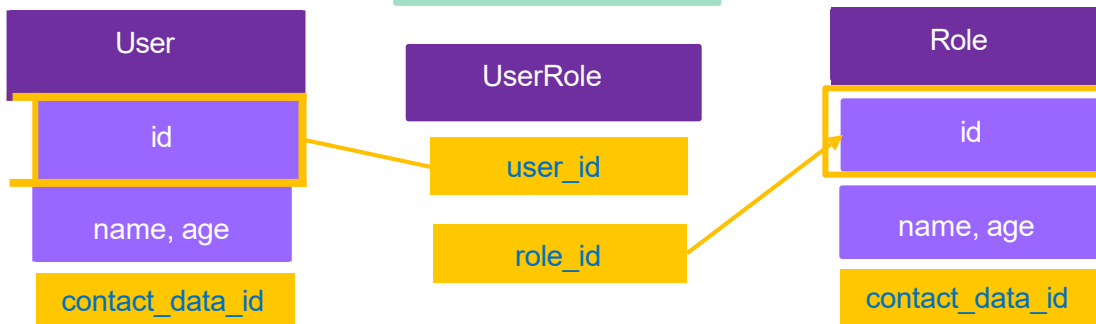
One-to-One



One-to-Many



Many-to-Many



RDBMS: ACID

Atomicity: A transaction happens completely or not at all. **Example:** Bank Transfer

Consistency: A transaction must keep the database in a valid state. **Example:** Student age must be greater than 0

Isolation: Simultaneous transactions should not interfere with each other.

Durability: Once committed, data is permanently saved.

The challenge of big data

Nowadays, big data has become tremendous due to:

- The explosion of social media sites such as Facebook and Twitter, user driven contents have increased rapidly. Hence, data, to be analysed and archived, have increased in volume and tend towards being semi structured and unstructured.
- Monitoring systems like GPS, sensors, automated trackers generate huge volumes of data on a regular basis ...

Horizontal scalability?



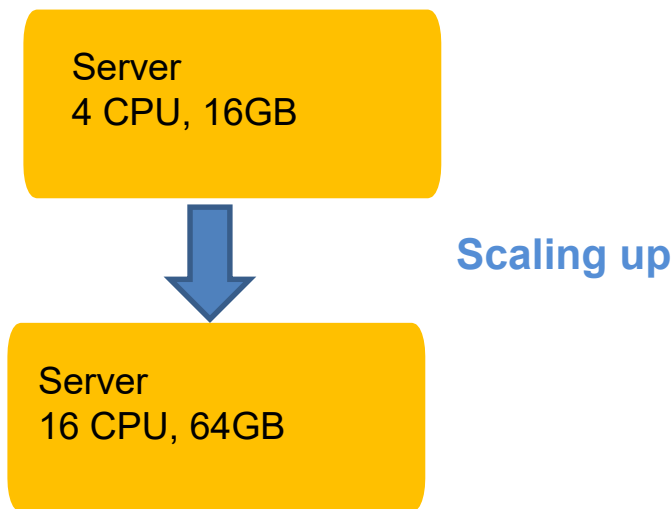
More volumes of data?
Unstructured data, performance
vs consistency

The factors above have made storing, analyzing, managing and archiving these big data a challenging task!

Vertical Scaling

As to handle the larger volumes of data, vertical scaling is used in RDBMS(Relational Database Management System).

It means increasing the capacity of a single database server by adding more hardware resources, including: CPU cores, More RAM, Faster storage, Better network capacity



The remedy to these challenges is **NOSQL**

What is NOSQL?

- It stands for **N**ot **O**nly **S**QL (**NOSQL**). It is a new set of database that is not based on the **R**elational **D**atabase **M**anagement **S**ystems principles.
- NoSQL was introduced by Carl Strozzi on **1998** to name his file-based database.
- In **2000**- Graph database Neo4j was launched.
- **2004** – Google BigTable was launched.
- **2005**- CouchDB is launched.
- **2007**- The research paper on Amazon Dynamo is released.
- **2008**- Facebooks open sources the Cassandra project.
- **2009**- The term NoSQL was reintroduced.

What is NOSQL? (Con't)

NoSQL databases have the following features in common:

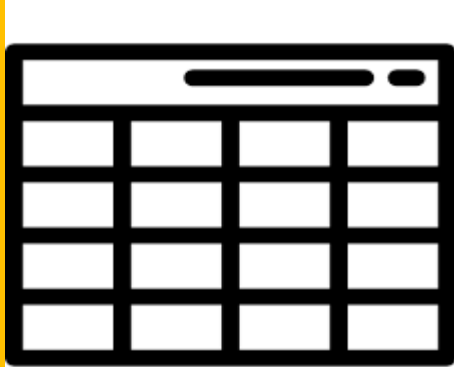
- They do not use relational data model
- Do not follow a strict schema
- They run well on clusters
- Most of them are open source
- They are made for the new generation of web applications

What is NOSQL? (structure)

The structure is as follows: for ex fb uses graphs based to store firends with their friends ...

Structure

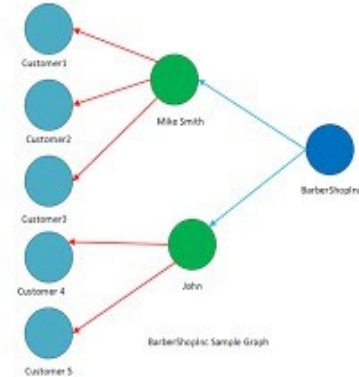
Tables



Documents

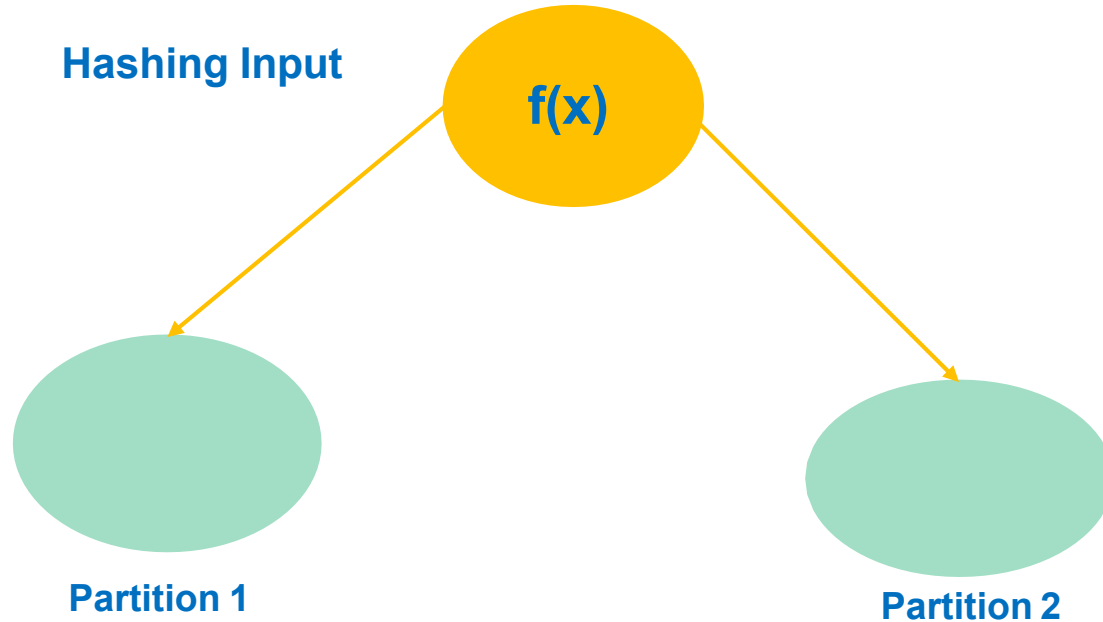


Graphs



What is NoSql? storage

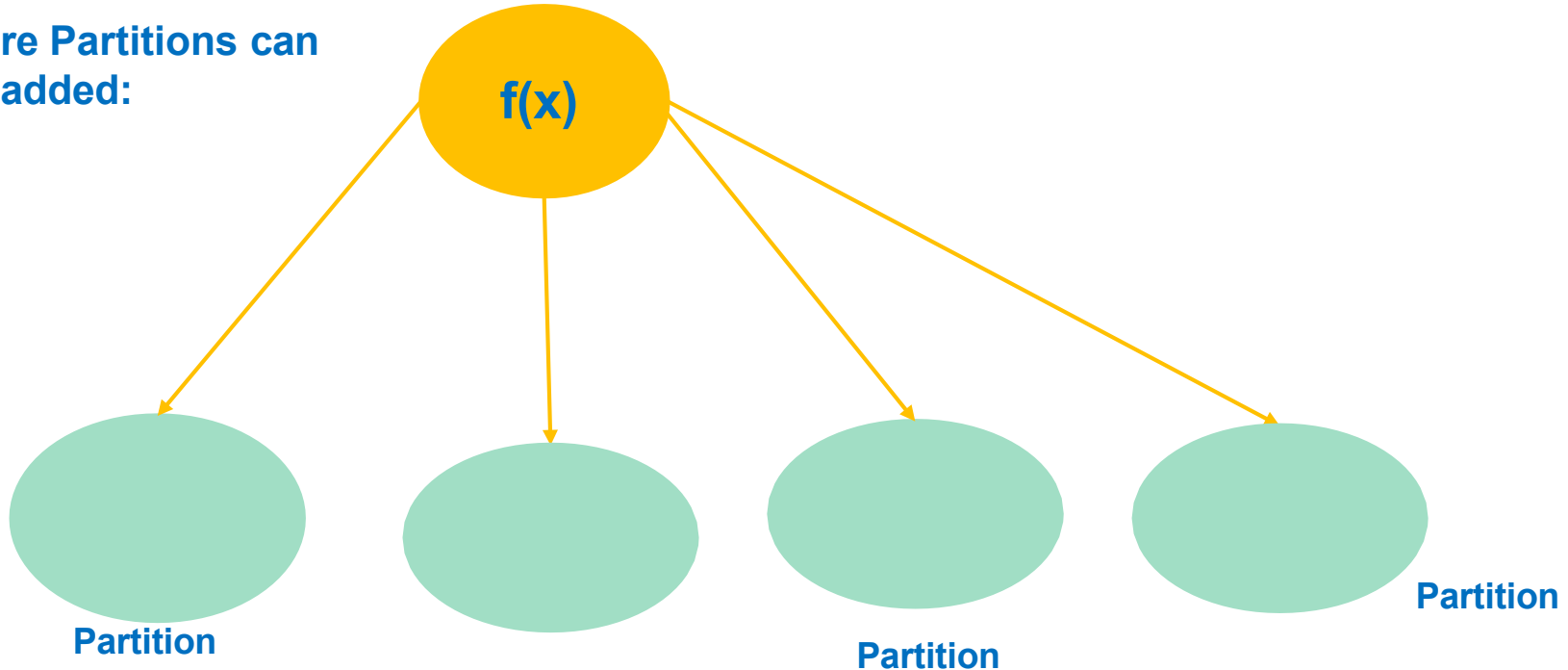
Data is stored in partitions using a hashing function that takes the key as a paramater. Assuming we have two partitions only:



What is NoSql? scaling

More partitions can be added to NoSQL database in order to accomodate more data => **Horizontal Scaling**

More Partitions can be added:



Advantages and disadvantages of NoSQL

Advantages

- High performance and scalability
- Availability and flexibility
- Open source and schema less

Disadvantages

- Lack of standardization
- Consistency issues
- Limited query capabilities

The difference between RDBMS and NoSQL Databases

	RDBMS	NoSQL Databases
Data model	Data is stored using a relational model with rows containing records and columns storing the attributes.	It does not use relational model
Schema	It follows a fixed , strict and already defined schema. Columns are defined before data entry. Modifying the schema requires setting the DB offline and altering the entire database.	It follows flexible schema: columns can be added anytime.
Scalability	Supports vertical scalability (having more powerful servers, more cpu, ram more cost). Difficult to scale horizontally (adding more machines)	Supports horizontal scaling (adding commodity hardware: cheap servers) => as it supports distributed architecture

The difference between RDBMS and NoSQL

	RDBMS	NoSQL Databases
Type /structure	Table based databases	Can be document based, graph based, key value pairs
Usage	Is used in case of having complex queries like joins	Used when queries are not complex (especially read operations) granting higher performance as it handles large volume of data
Open source	Is a mix of open source products like MySQL and commercial ones like Oracle database	NoSQL databases have been always open source
The followed principle	The ACID model: Atomicity, Consistency, Isolation, and Durability (ACID) compliant.	The BASE model: Basically available, Soft state and eventually consistent

When to use SQL or no SQL?

SQL

- When access patterns **are not** defined.
- When users want to perform flexible queries/relational queries (like joins).
- When it is required to enforce fields constraints (data type...)

NoSQL

- When access patterns are defined.
- When the use of complex queries is not required.
- When you require high performance and low latency.

Types of NoSQL databases?(Con't)

This is why NOSql databases were introduced. There exist four types:

Key-Value Database

Document Database
(document-oriented)

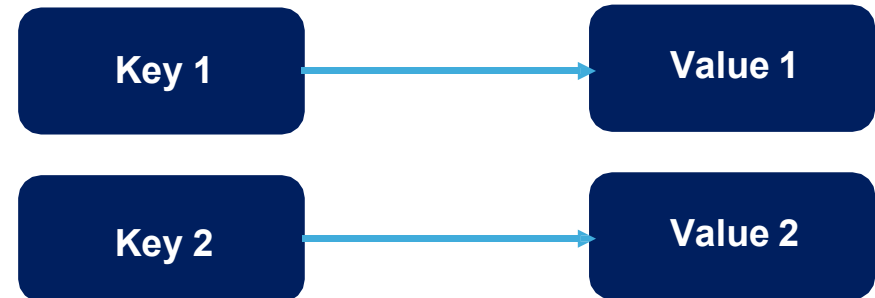
Column based data
stores:

Graph Database

Types of NoSQL databases: Key-Value DB

Definition:

- The simplest NoSQL database.
- Every single item in the DB is stored as an attribute name or “**key**” along with its **value**.
- Data is stored as a collection of **key/value** pairs.
- The key in a key-value pair must be **unique**.
- The values can be integer, string or complex data types.
- Data is retrieved via an exact match of the key.
- New types of data can be easily added to the database as new key-value pairs.



Types of NoSQL databases: Key-Value DataBase

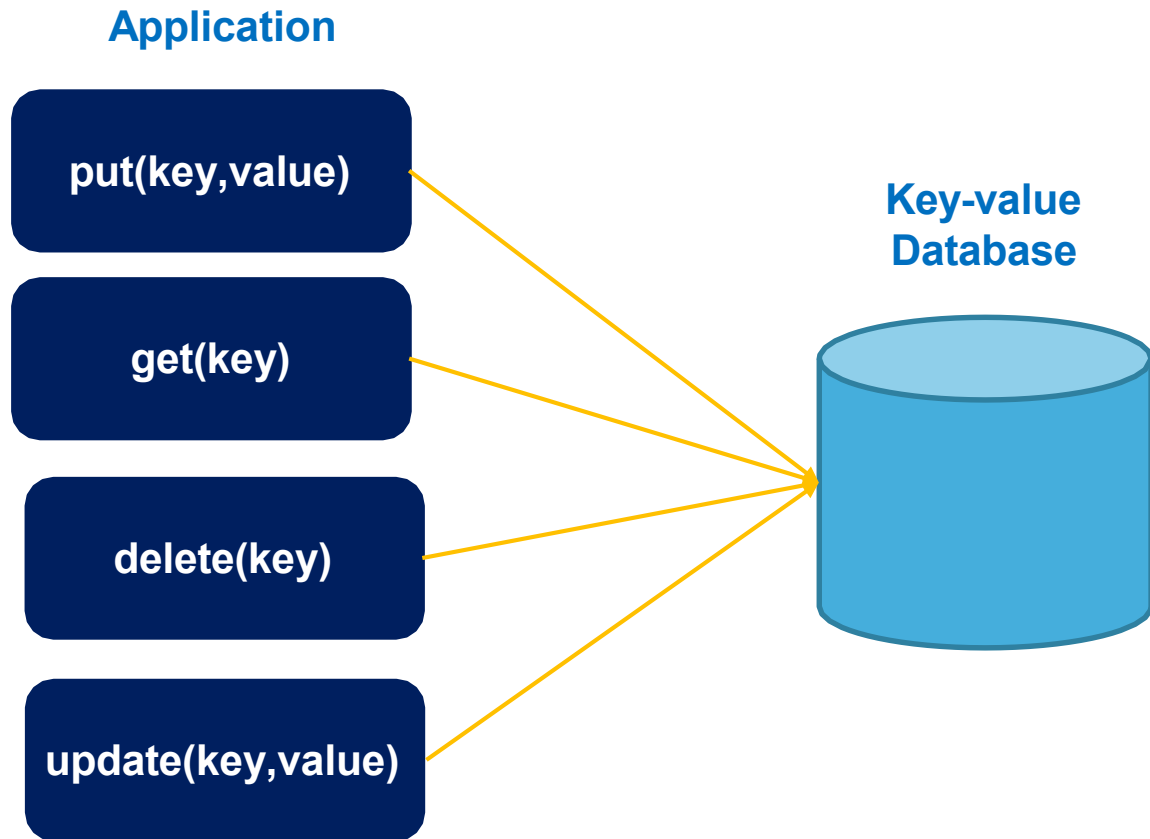
Operations:

Here are the operations that can be performed on a key-value database are:

- **put a new (key, value)**: Insert the value associated with a key.
- **get(key)**: a key is passed to retrieve the value associated with the key.
- **delete(key)**: removes the value associated with a key.
- **update (key, value)**: Update the value associated with a key.

Types of NoSQL databases: Key-Value DataBase

Operations:



Types of NoSQL databases: Key-Value DataBase

Advantages:

- It is very easy to use. Due to the simplicity of the database, data can accept any kind, or even different kinds when required.
- Its response time is fast due to its simplicity, given that the remaining environment near it is very much constructed and improved.
- Key-value store databases are scalable vertically as well as horizontally.

Types of NoSQL databases: Key-Value DataBase

Use cases:

- **In-memory data caching:** it is an effective cache mechanism for frequently accessed but rarely updated data to accelerate the response of applications => faster read and write. Hazelcast is an example of a technology that provides an in-memory key-value store for fast data retrieval
- **User preference storage and personalisation:** it can be used to provide recommendations and store personalized lists of items for individual clients. For instance e-commerce platforms, they can customize/update ads accordingly.

Types of NoSQL databases: Key-Value DataBase

Known key value DBs:



amazon
DynamoDB

◁EROSPIKE



Couchbase

Types of NoSQL databases?

Document Database (document-oriented):

- Document databases are considered to be non-relational or nosql databases.
- Instead of storing data in fixed rows and columns, document databases use flexible documents. For a document database, you have data store like JSON object. You do not require to define which make it **flexible**.
- Document databases are the most popular alternative to tabular, relational databases.

Hence, what is a document?

Types of NoSQL databases?

Document Database (document-oriented):

- A **document** is a record in a document database. A document typically stores information about one object and any of its related metadata.
- Documents store data in **field-value** pairs. The values can be a variety of types and structures, including strings, numbers, dates, arrays, or objects. Documents can be stored in formats like JSON, and XML.

Types of NoSQL databases?

Document Database (document-oriented):

Here is a JSON document that stores information about a user named **Tom**.

```
{
  "_id": 1,
  "first_name": "Tom",
  "email": "tom@example.com",
  "cell": "765-555-5555",
  "likes": [
    "fashion",
    "spas",
    "shopping"
  ],
  "businesses": [
    {
      "name": "Entertainment 1080",
      "partner": "Jean",
      "status": "Bankrupt",
      "date_founded": {
        "$date": "2012-05-19T04:00:00Z"
      }
    },
    {
      "name": "Swag for Tweens",
      "date_founded": {
        "$date": "2012-11-01T04:00:00Z"
      }
    }
  ]
}
```

Types of NoSQL databases?

Document Database (document-oriented):

Here is a JSON document that stores information about a user named **Tom**.

But, what happens in case of having many Users?



In this case, we talk about collections.
So what is collection?

```
{
  "_id": 1,
  "first_name": "Tom",
  "email": "tom@example.com",
  "cell": "765-555-5555",
  "likes": [
    "fashion",
    "spas",
    "shopping"
  ],
  "businesses": [
    {
      "name": "Entertainment 1080",
      "partner": "Jean",
      "status": "Bankrupt",
      "date_founded": {
        "$date": "2012-05-19T04:00:00Z"
      }
    },
    {
      "name": "Swag for Tweens",
      "date_founded": {
        "$date": "2012-11-01T04:00:00Z"
      }
    }
  ]
}
```


Types of NoSQL databases?

Document Database (document-oriented):

- A **collection** is a group of documents. Collections typically store documents that have **similar** contents (similar to a table in RDBMS).
- Not all documents in a collection are required to have **the same fields**, because document databases have a **flexible schema**.
- Note that some document databases provide schema validation, so the schema can optionally be locked down when needed.

NoSQL databases: Document based Database

The structure of the database:

Database

Collection

Collection

Collection

Document#1
Key:Value

Document#2
{Key:Value,
Key:Value}

Document#3
Key:Value

Document#4
Key:Value

NoSQL databases: Document based Database

Example:

- For example, following up the document for the user **Tom**, it can be added to a collection named **users**.
- Also, if considering a new user called **Donna**, it can be also added to the collection **users**.
- Hence, the **users Tom** and **Donna** belong to the same collection.
- Note that the document for **Donna does not contain the same fields as the document** for Tom. The users collection is leveraging a **flexible schema** to store the information that exists for each user.

```
{
  "_id": 2,
  "first_name": "Donna",
  "email": "donna@example.com",
  "spouse": "Joe",
  "likes": [
    "spas",
    "shopping",
    "live tweeting"
  ],
  "businesses": [
    {
      "name": "Castle Realty",
      "status": "Thriving",
      "date_founded": {
        "$date": "2013-11-21T04:00:00Z"
      }
    }
  ]
}
```

Why is it easier to work with documents than tables

- Developers commonly find working with data in documents to be **easier and more intuitive** than working with data in tables.
- Documents map to data structures in most popular programming languages. Developers don't have to worry about manually **splitting** related data across multiple tables when storing it or **joining** it back together when retrieving it.
- JSON documents allow you to embed related documents allowing easy fetching of data in **one single call**. Therefore, storing data in the JSON format is truly a natural way to store information unlike the flat data structure of rows and columns.

NoSQL databases: Document based Database

Example of storing a user Tom in RDBMS and document based database:

Let's take another look at a document for a user named **Tom**:

- All of the information about Tom is stored in a single document.

```
{
  "_id": 1,
  "first_name": "Tom",
  "email": "tom@example.com",
  "cell": "765-555-5555",
  "likes": [
    "fashion",
    "spas",
    "shopping"
  ],
  "businesses": [
    {
      "name": "Entertainment 1080",
      "partner": "Jean",
      "status": "Bankrupt",
      "date_founded": {
        "$date": "2012-05-19T04:00:00Z"
      }
    },
    {
      "name": "Swag for Tweens",
      "date_founded": {
        "$date": "2012-11-01T04:00:00Z"
      }
    }
  ]
}
```

NoSQL databases: Document based Database

Example (con't):

Now let's consider how we can store that same information in a relational database(RDBMS).

We'll start by creating a table **User** that stores the basic information about the user:

ID	first_name	email	cell
1	Tom	tom@example.com	564-777-521

NoSQL databases: Document based Database

Example (con't):

A user can like many things (meaning there is a one-to-many relationship between a user and likes), so we will create a new table named "Likes" to store a user's likes. The Likes table will have a foreign key that references the ID column in the User's table.

ID	user_id	like
10	1	fashion
11	1	travel
12	1	shopping

NoSQL databases: Document based Database

Example (con't)

Similarly, a user can run many **businesses**, so we will create a new table named "Business" to store business information. The Business table will have a foreign key that references the ID column in the Users table

ID	user_id	name	partner	status	date_founded
20	1	Entertainment 1080	Jean	Bankrupt	2011-05-19
21	1	Phones shop	NULL	NULL	2012-11-01

NoSQL databases: Document based Database

CRUD operations:

- **Create:** Documents can be created in the database. Each document has a unique identifier.
- **Read:** Documents can be read from the database. The API or query language allows developers to query for documents using their unique identifiers or field values. Indexes can be added to the database in order to increase read performance.
- **Update:** Existing documents can be updated — either in whole or in part.
- **Delete:** Documents can be deleted from the database.

NoSQL databases: Document based Database

Use cases: both transactional and analytical applications:

- Mobile applications
- Processing of payment operations
- Customer data management and personalization
- Internet of Things (IoT)
- Product catalogs and content management
- Operational analytics
- Real-time analytics

Examples of document-based databases:

MongoDB, OrientDB, Apache CouchDB, IBM Cloudant, CrateDB, BaseX, and many more



NoSQL databases: Graph based database

Definition:

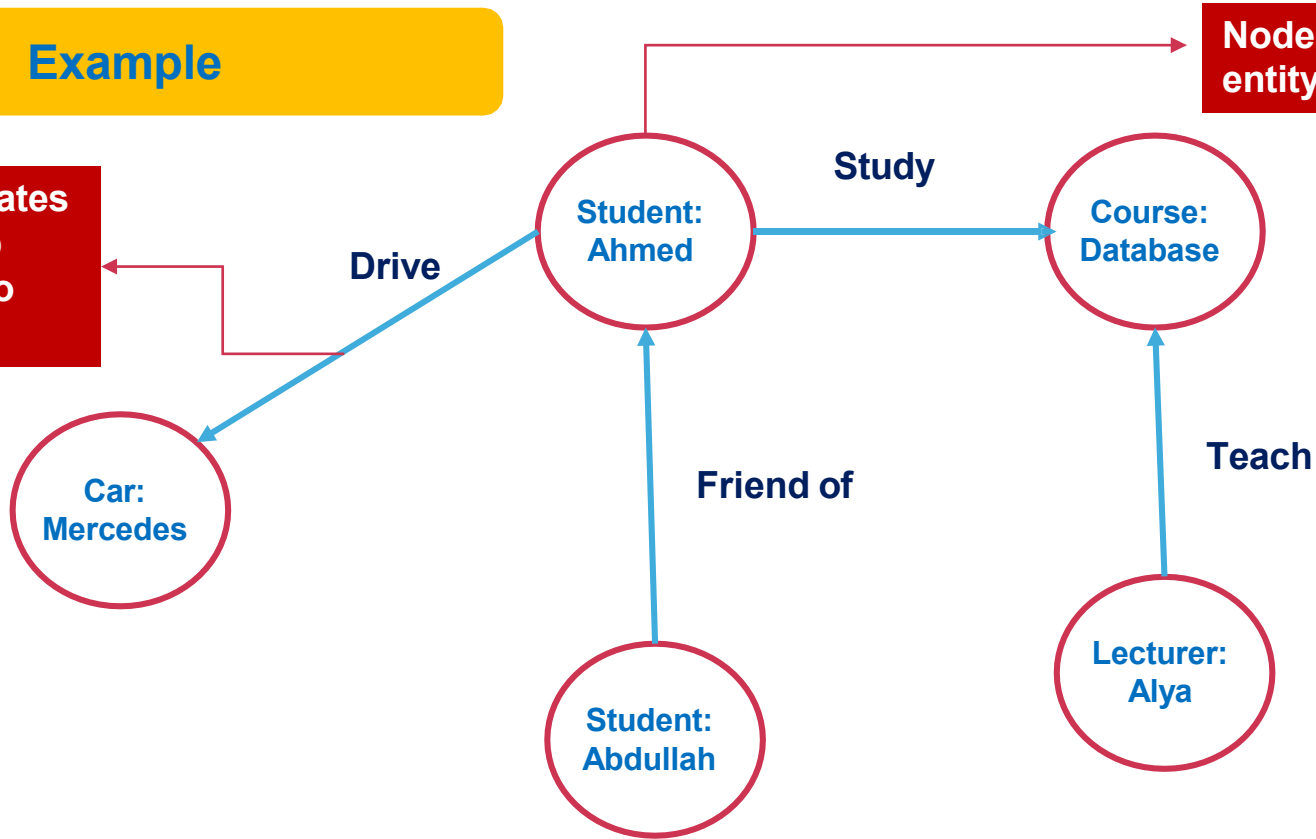
- Data is organized in the form of a **graph(directed graph with edges)**.
- A **graph** db contains a collection of nodes and edges.
- A **node** represents an entity and an **edge** represents the relationship between two entities.
- Like many different NoSQL databases these data models don't have any schema as it is important because schema makes the model well and good and easy to edit.

NoSQL databases: Graph based database

Example

Edge: illustrates
relationship
between two
entities

Node: represents an
entity



NoSQL databases: Graph based database

Use cases:

- **Social media analysis:** Graph databases are commonly used to analyse social networks. A heavy volume of user accounts(nodes) and multi-dimensional connections. A graph analysis for a social network can determine:
 - How active are users? (number of nodes)
 - Which users have the most influence? (density of connections) ...
- **Credit card fraud:** As a means of fraud detection, pattern identification is often the first line of defense thanks to graphs based databases.

Types of NoSQL databases?

Graph based Database: example



Types of NoSQL DBs: Column based databases:

Definition:

- A **column** store database is a type of database that stores data using a column oriented model.
- Columns store databases uses a concept called **keyspace**.
- A **keyspace** is like a schema in the relational model. The keyspace contains all the **column families** (like tables in case of relational model).

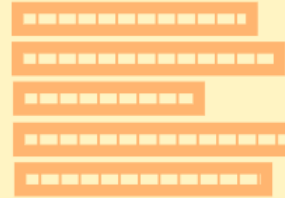
Types of NoSQL DBs: Column based databases:

Definition:

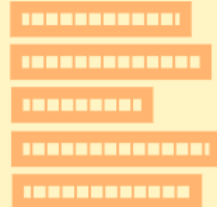
- Columns store databases uses a concept called **keyspace**.
- A **keyspace** is like a schema in the relational model. The keyspace contains all the **column families** (like tables in case of relational model).

Keyspace

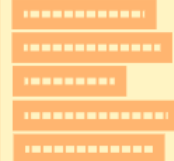
Column Family



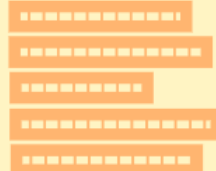
Column Family



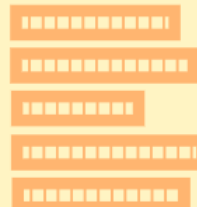
Column Family



Column Family



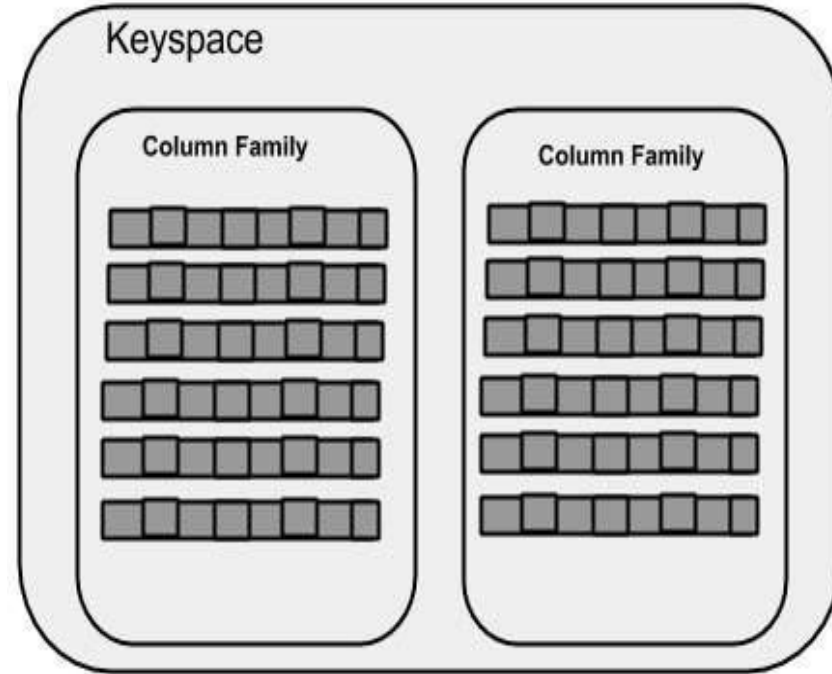
Column Family



Types of NoSQL databases: Column based

Defintion:

- The column families are defined initially, **the columns are not**. Columns can be added to any **column family at any time**.
- Unlike relational tables, column family's schema is **not fixed**, and this type of database **does not force** individual rows to have all the columns.



Example of column family UserProfile

UserProfile

Bob	emailAddress	gender	age
	bob@example.com	male	35
	1465676582	1465676582	1465676582
Britney	emailAddress	gender	
	brit@example.com	female	
	1465676432	1465676432	
Tori	emailAddress	country	hairColor
	tori@example.com	Sweden	Blue
	1435636158	1435636158	1465633654

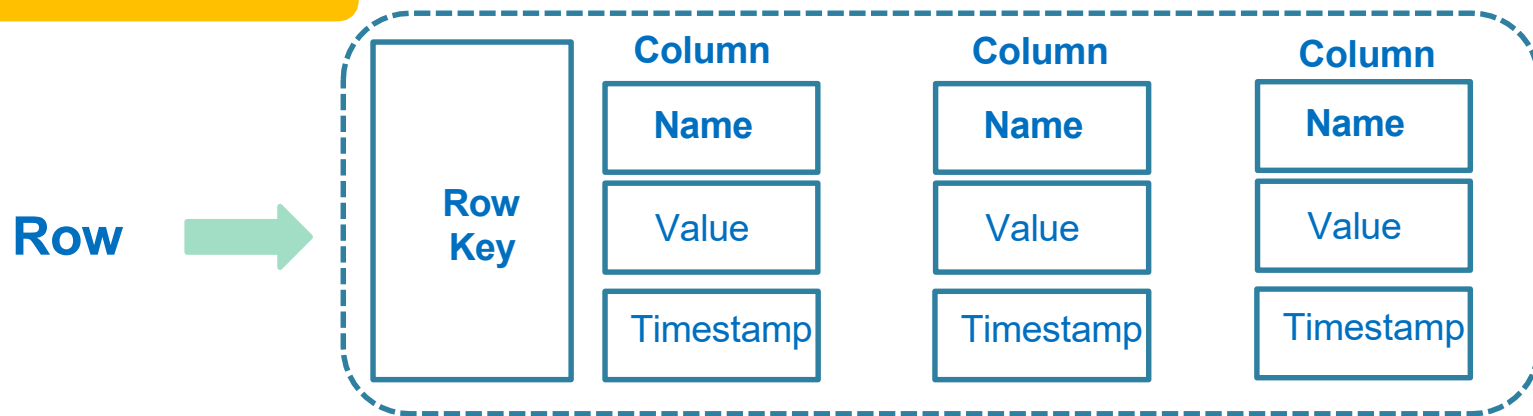
Column-based nosql approach

Column based databases:

- Column family consists of **multiple rows**.
- Each row can contain a different number of columns to the other rows. And the columns don't have to match the columns in the other rows (they can have different column names, data types).
- Each column is contained to its row. It doesn't span all rows like in a relational database. Each column contains a name/value pair, along with a timestamp(date of insertion of the data).

Column-based NoSql approach databases

Row Structure:



- Row Key. Each row has a unique key, which is a unique identifier for that row.
- Column. Each column contains a name, a value, and timestamp.
- Name. This is the name of the name/value pair.
- Value. This is the value of the name/value pair.

DataBases using the column based approach

Comparison between RDBMS and Column based DBs

Column based	RDBMS
Keyspace	database
Column family	table
Record	Record
Column that vary amongst records	Columns are the same among all records

Column-based nosql approach

Use cases:

- **The storage of error reports:** The flexibility of these DBs allow the optimal storage of error reports generated by several applications, where each one of them can use the suitable group's column.
 - **Blogs data management:** Blogs use these DBs to store publications, their links, and the associated comments.
 - **Number of visitors reports:** The systems for counting the number of visitors use these kinds of databases thanks to the dynamic nature of their columns. This is to allow the classification per page/site.
- Massive datasets (billions of rows), Time-series data, IoT sensor data, Event logging, Real-time analytics**

DataBases using the column based approach

Column based databases:

HBase, Cassandra, HBase, Hypertable are NoSQL query examples of column- based databases



Recap

NoSQL Type	Advantages	Disadvantages	Best Use Cases
Document-Based	Flexible schema, stores complex/nested data naturally, easy to map to application objects, supports rich queries	Data duplication, joins are limited, complex relationships can be difficult	User profiles, e-commerce catalogs, content management, web applications
Column-Based (Wide-Column)	Highly scalable, excellent for huge datasets, fast writes and reads on specific columns, good for distributed systems	More complex data modeling, limited joins, less intuitive than SQL	Big data, time-series data, IoT, event logging, analytics
Graph-Based	Excellent for highly connected data, very fast relationship traversal, intuitive representation of networks	Not ideal for large aggregations or tabular data, or statistics. It can be harder to scale horizontally, except infinigraph cases	Social networks, recommendation systems, fraud detection, network analysis
Key-Value	Extremely fast, simple design, highly scalable, low latency	Queries usually limited to key lookup, little support for relationships or complex searches	Caching, session storage, shopping carts, real-time applications